

Homework Assignment – Advanced AI Phishing Detection System

This assignment is built upon the provided phishing detection lab. The objective is to extend, analyze, and critically improve the system. Students are expected to demonstrate both implementation skills and analytical reasoning. Each task must be clearly documented and justified.

Learning Objectives

By completing this assignment, students should be able to:

- Design and evaluate feature engineering strategies
- Compare machine learning models in a cybersecurity context
- Analyze model limitations (false positives / false negatives)
- Extend a real-world application
- Critically assess AI-based security systems

Provided Material

- ai_phishing.zip
- dataset_classroom.csv / dataset_homework.csv
- Sample input files

Required Tasks

1. Feature Engineering (Core Task)

Modify `feature_extraction.py` and implement at least three new meaningful features.

Steps:

- Identify limitations in existing features
- Design new features (e.g., advanced URL analysis, domain similarity, linguistic tone)
- Implement features in `compute_handcrafted_features()`
- Justify why each feature improves detection
- Show before/after impact on model performance

2. Enhancement of Phishing Detection Logic

Extend `phishing_cues()` with new detection rules.

Steps:

- Add at least three new pattern categories (e.g., financial pressure, impersonation signals)
- Improve explanation messages returned to the user

- Ensure cues are meaningful and not redundant
- Demonstrate how these affect risk scoring

3. Model Retraining and Evaluation

Retrain the model using `train_model.py`.

Steps:

- Train baseline model
- Train enhanced model with new features
- Compare metrics (precision, recall, F1-score)
- Interpret results
- Explain any trade-offs (e.g., increased recall vs false positives)

4. Model Replacement and Comparison

Replace the classifier with an alternative model.

Steps:

- Select a different model (e.g., Random Forest, SVM)
- Justify your choice
- Train and evaluate
- Compare with baseline model
- Explain which model is more suitable and why

5. Error Analysis (Critical Task)

Perform a detailed analysis of model errors.

Steps:

- Identify if there are false positives and two false negatives
- Examine why the model failed in each case
- Link failures to missing features or weak patterns
- Propose concrete improvements

6. Application Extension

Modify the application (`app.py` and/or UI).

Steps:

- Improve the explanation of predictions
- Add at least one new user-visible feature (e.g., detailed breakdown, warning messages)
- Ensure outputs are interpretable
- Optionally improve UI clarity

Deliverables

Students must submit:

- Updated source code
- A structured report including:

- Description of implemented features
- Model comparison and justification
- Error analysis
- Discussion of system limitations
- Screenshots of system outputs
- Comparative results (before vs after)

Optional Advanced Tasks

Students aiming for higher grades may complete one or more of the following:

- Integrate a transformer-based model (e.g., BERT)
- Implement a weighted or adaptive risk scoring system
- Introduce a new dataset and evaluate generalization
- Design adversarial examples to test robustness
- Extend system to support additional input formats